Exact Nearest-Neighbor Search with Composable Lower Bounds: Self-Configuring Hierarchical Pruning, In RAM and Out of Core

Tai-Wu Chiang
twchiang@utexas.edu
Independent Researcher
California, USA

ABSTRACT

Many applications cannot tolerate a missed true neighbor. Examples include deduplication, medical and legal retrieval, and scientific analysis. These applications need *exact* nearest-neighbor search. The standard tool for fast exact search is a cheap lower bound on each candidate's distance: if the bound already exceeds the best distance found so far, the candidate can be discarded without computing its full distance. This paper builds such bounds from truncated PCA projections. These bounds are tight when variance concentrates in a few dimensions. We further observe that they belong to a general family: any region that provably contains a set of points yields an exact lower bound, and different region shapes are tight for different data structures. We therefore compose two complementary families into a single exact cascade. The first is PCA truncation, which is tight for linearly correlated data. The second is a triangle-inequality ball bound around k-means centroids, which is tight for clustered data. The ball bound is exact for *any* choice of centers, so clustering quality affects only speed, never correctness.

Neither bound family helps on all data. The index therefore configures itself from a small sample. The cascade widths are chosen from the eigenvalue curve; fixed widths tuned for 128 dimensions misread 1536-dimensional OpenAI embeddings as unprunable, while widths placed at fixed variance targets recover them. A three-tier diagnostic then decides which levels to build: the eigenvalue curve and two cheap statistics nominate, and a measured trial of the real cascade decides. We report results in hardware-independent work units alongside wall-clock time, because wall-clock proved regime-dependent while work counts never moved. On four real datasets spanning 96 to 1536 dimensions (SIFT1M, GIST1M, DBpedia-OpenAI-1M, and DEEP10M), exact 1-NN search performs 3.9–20.8× less work than a full scan. Against a NumPy brute-force scan, the in-RAM wall-clock gain is 1.2× at 96 dimensions, 2.6× at 128, and about 6× at 960 and 1536; a single-thread FAISS scan still wins in RAM at 128 dimensions. Exact k-NN generalizes the pruning radius to the k-th order statistic; at $k = 10$ it wins on three of the four datasets, and the PCA-only cascade loses in RAM at $k = 100$. Out of core, the fetch set concentrates in few k-means clusters (27 of 1024 on average on the embeddings). A measured two-tier layout with `O_DIRECT` cold reads answers exact queries 2–17× faster than our full disk scan, or 1.1–8.8× faster than the device's sequential-bandwidth limit for any scan, and reads 18–730× fewer bytes. We also compare against ADSampling, the state-of-the-art statistical pruner. In RAM the work counts are mixed. ADSampling pays a 32-dimension floor on every vector, while our cascade fetches only 0.06–1.2% of the vectors; an equal-residency out-of-core comparison remains future work. Exactness is verified on every query of every in-RAM experiment. All code, diagnostics, and experiments are public.

PVLDB Reference Format:
Tai-Wu Chiang. Exact Nearest-Neighbor Search with Composable Lower Bounds: Self-Configuring Hierarchical Pruning, In RAM and Out of Core. PVLDB, 20(1): XXX-XXX, 2027.
doi:XX.XX/XXX.XX

PVLDB Artifact Availability:
The source code, data, and/or other artifacts have been made available at https://github.com/taiwuchiang-gmail/hierarchical-pca-search.

1 INTRODUCTION

Finding the exact nearest neighbor in a large, high-dimensional dataset is limited by the cost of computing full distances. Approximate methods such as LSH [7], HNSW [11], and IVF [8] trade recall for speed. However, some applications cannot accept a missed true neighbor. Examples include deduplication, medical and legal retrieval, and scientific analysis. For these applications, the alternative is exact pruning in the GEMINI tradition [4]: compute a cheap *lower bound* on each candidate's distance, and discard the candidate if the bound already exceeds the best distance found so far. Our starting point is such a bound from truncated PCA projections (§3.1). After the data is rotated onto its principal axes, the distance between k-dimensional prefixes is a lower bound on the true distance. A cascade of progressively finer prefixes, seeded by a dynamic Top-K probe, prunes over 99.9% of the SIFT1M dataset and returns provably exact results.[1]

That bound sees only linear structure. PCA truncation is tight on average when variance concentrates in a few principal directions, which means linearly correlated data. Clustered data can behave very differently. Its eigenvalue spectrum can be flat, so the eigen curve predicts a loose bound. Yet the same data is highly compressible in another sense: points huddle near a modest number of centroids, and a bound built on those centroids costs one subtraction per point rather than a multiply-add per coordinate. No single bound family serves all data. Worse, a practitioner has

This work is licensed under the Creative Commons BY-NC-ND 4.0 International License. Visit https://creativecommons.org/licenses/by-nc-nd/4.0/ to view a copy of this license. For any use beyond those covered by this license, obtain permission by emailing info@vldb.org. Copyright is held by the owner/author(s). Publication rights licensed to the VLDB Endowment.
Proceedings of the VLDB Endowment, Vol. 20, No. 1 ISSN 2150-8097.
doi:XX.XX/XXX.XX

[1] Preliminary versions of this work appeared as two self-archived reports (doi:10.5281/zenodo.21387358, doi:10.5281/zenodo.22199955). This paper merges, extends, and supersedes them. The k-NN generalization, the eigen-curve width selection, three of the four datasets, the measured out-of-core results, and the ADSampling baseline are new.

no way to know, before building an index, which family (if any) their data supports.

This paper makes two moves. First, **exact bounds compose**. The maximum of valid lower bounds is itself a valid lower bound, so bound families with different blind spots can share one cascade. We add a triangle-inequality *ball bound* around k-means centroids. It is the clustered-data complement of PCA truncation, and it is exact for *any* choice of centers: clustering quality affects only pruning power, never correctness. Second, **the composition is chosen by measurement**. A three-tier diagnostic runs on a sample in seconds. It rates the PCA levels from the eigenvalue decay, and it detects structure beyond the covariance with two cheap statistics. We show that statistics alone mispredict in both directions. The diagnostic therefore settles the question with a measured trial of the real cascade, and emits an ADD/SKIP verdict for each level.

There is also an economic motivation. An exhaustive flat scan requires the entire dataset to be resident in RAM, which is the most expensive byte in the machine. RAM costs roughly an order of magnitude more than NVMe per byte. The pruning cascade needs only its sketch resident: the first max(levels) rotated coordinates plus 16 bytes of per-point metadata. On our four datasets this is 1.8–7.7× less RAM than the data (for example, 272 versus 512 bytes per vector on SIFT1M, and 496 versus 3,840 on GIST1M), with exactness preserved. Quantizing the sketch with conservative cell bounds, as the VA-file does [14], would shrink it further; we leave that to future work.

Our contributions:

(1) **Exact hierarchical pruning from composable bounds.** A PCA-prefix cascade with a dynamic Top-K pruning radius, unified with the triangle-inequality ball bound under one containing-region principle. The two families exploit different structure: linear correlation and clustering (§3).
(2) **A composed exact cascade, generalized to k-NN.** The cascade combines a ball level and the PCA levels, with a radius re-probe. Exact k-NN uses the k-th order statistic over evaluated points as the pruning radius. In RAM, the storage overhead is about 8 bytes per point (§5).
(3) **Self-configuration that includes the cascade widths.** Widths are placed at fixed cumulative-variance targets on the eigenvalue curve; a fixed ladder is silently calibrated to one dimensionality and misreads others. The three-tier funnel then rules on each level, with verdicts derived from measured work ratios. Negative verdicts are included (§4).
(4) **Full-scale evidence on four real datasets** (96–1536 dimensions, 10^6–10^7 points), reported in hardware-independent work units. Wall-clock serves as cost-model validation. Negative controls and sample-to-full-scale checks are included (§6).
(5) **A measured out-of-core design.** We measure the fetch set's cluster locality, build a two-tier on-disk layout, and run cold-read benchmarks under `O_DIRECT`. Exact search is 2–17× faster than our full disk scan, and 1.1–8.8× faster than the device's sequential-bandwidth limit for any scan (§6.6).
(6) **An ADSampling baseline** in a shared work currency. It locates the in-RAM boundary between statistical and deterministic pruning (§6.7).

2 BACKGROUND AND RELATED WORK

Lineage. Neither bound family is new, and we do not claim otherwise. Lower-bounding by dimensionality reduction descends from the GEMINI framework [4] and the VA-file [14]. Pruning by the triangle inequality around reference points is the basis of ball trees [12], vantage-point trees [15], GNAT [1], and Elkan's accelerated k-means [3]. The approximate cousins are FAISS IVF [8], which uses the same k-means geometry without exactness, and OPQ [6], which learns a global rotation. Its locally optimized variant LOPQ [9] rotates each cell separately, which corresponds to the ellipsoid rung of the shape ladder in §3. Disk-resident ANN systems such as DiskANN [13] and SPANN [2] target the same memory-economics regime as we do, but they are approximate. The closest method in mechanism is ADSampling [5]. It also evaluates incremental prefix distances after a rotation, but it replaces the deterministic bound with a sequential hypothesis test under a *random* rotation. PDX [10] accelerates this style of pruning with a dimension-major layout. We measure against ADSampling directly in §6.7. Our contribution is threefold: the *exact composition* of the two bound families in one cascade, the *measured self-configuration* including the cascade widths themselves, and a diagnostic that also tells the practitioner when to walk away.

3 COMPOSABLE EXACT BOUNDS

3.1 The PCA Truncation Bound

Let $X, Y \in \mathbb{R}^d$ and let X', Y' be their images under the orthogonal rotation onto the dataset's principal axes. Rotation preserves the squared Euclidean distance, $L_2^2(X,Y) = L_2^2(X',Y') = \sum_{i=1}^{d}(x_i' - y_i')^2$. Let T_k retain the first k coordinates.

LEMMA 3.1. *For any truncation width $k < d$,*

$$L_2^2\big(T_k(X'), T_k(Y')\big) \le L_2^2(X,Y).$$

PROOF. The true squared distance is a sum of d non-negative terms; the truncated distance is the sum of the first k of the same terms, and $\sum_{i=k+1}^{d}(x_i' - y_i')^2 \ge 0$. □

If the prefix distance to a candidate already reaches the current search radius r, the candidate is discarded without computing its full distance. This is the GEMINI no-false-dismissal condition [4]. Two choices turn the bound into an algorithm. The first choice is the rotation. Any orthogonal rotation satisfies Lemma 3.1. PCA makes the bound *tight*: by the Karhunen–Loève theorem, it packs the maximum variance into the first k coordinates, and therefore the largest expected share of the squared distance between two random points (quantified in Eq. (3)). In other words, exactness is unconditional but quality is not. This separation recurs throughout the paper. The second choice is the source of the radius. Instead of a static threshold, a Top-K probe at the coarsest width computes exact distances for the $K = 50$ most promising candidates and sets r to their minimum. Because r is the exact distance to an actual

Table 1: Shape families and their exact bounds. Each is tight for a different data structure; each mirrors a quantizer family.

Shape	Bound cost	Tight for	Quantizer twin
ball	$\lvert \lVert q-c \rVert - d_p \rvert$	clusters	VQ
ellipsoid	per-axis scaled	stretched clusters	LOPQ
slab	projection residual	local low rank	local PCA
cone×annulus	closed form	radial/gain data	gain–shape

data point, it is at least the true nearest-neighbor distance; and because the prefix distance is a lower bound, pruning against such an r never discards the true neighbor. The radius is therefore safe from the first comparison on, and no prior knowledge of the neighbor distance is needed. A cascade of progressively finer prefixes then re-prunes the survivors at each width. In our implementation each level recomputes its prefix distance from the first coordinate, and the work counts in §6 charge it that way. On SIFT1M, widths $(8, 16, 32, 64)$ discard all but an average of 564 of 10^6 vectors before any full-dimensional evaluation. Section 4 replaces these fixed widths with widths derived from the data's own eigenvalue curve.

3.2 The Containing-Region Principle

Let $R \subseteq \mathbb{R}^d$ be any region that provably contains a set of points C. Then for every $p \in C$ and any query q,

$$\lVert q - p \rVert \;\geq\; \mathrm{dist}(q, R), \tag{1}$$

so $\mathrm{dist}(q, R)$ is an exact pruning bound for every member of C at once. A bound family is therefore a *shape* family, and a shape is admissible when (a) containment is guaranteed and (b) $\mathrm{dist}(q, R)$ is cheap. Table 1 lists the natural ladder. Each geometric family is the twin of a quantizer family: a quantizer's codeword-plus-distortion model *is* a containing region. We use the first two rungs; the others are available under the same principle.

3.3 The Ball Bound

Cluster the dataset with k-means. For each point p assigned to centroid c, store the scalar $d_p = \lVert p - c \rVert$ once, offline. At query time, compute the k centroid distances $\lVert q - c_j \rVert$ (with $k \ll N$), and the triangle inequality gives, for every point,

$$\lVert q - p \rVert \;\geq\; \big|\, \lVert q - c_{a(p)} \rVert - d_p \,\big|, \tag{2}$$

where $a(p)$ is p's cluster. Evaluating (2) for all N points requires one gather and one subtraction per point—*no distance computations*. A coarser, per-cluster form uses the cluster radius $R_j = \max_{p \in j} d_p$: if $\lVert q - c_j \rVert - R_j$ exceeds the current radius, the entire cluster is eliminated in one comparison, which in an out-of-core layout skips a whole contiguous block (§5.4).

3.4 Exactness Is Unconditional; Quality Is Not

Inequality (2) is the triangle inequality, so it holds for *any* reference points. k-means enters only because it minimizes the mean d_p^2, which makes the bound *tight*. Three corollaries follow. First, stale centers remain correct as the dataset drifts. Pruning degrades gracefully, and re-clustering can wait until a diagnostic calls for it.

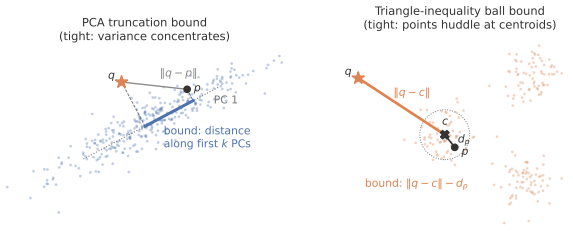


Figure 1: The two exact bound geometries. Left: the PCA truncation bound is the distance between the projections onto the first k principal components—tight when variance concentrates along few directions. Right: the ball bound $\lVert q - c \rVert - d_p$ from the triangle inequality—tight when points huddle near their centroids. The two bounds exploit different structure.

Second, insertion costs $O(kd)$, independent of N: assign the new point to its nearest existing centroid, store d_p, and raise that cluster's radius if needed. No re-clustering is required for correctness. Third, the composed index mirrors the PCA bound's property from §3.1. There, any orthogonal rotation is exact and PCA makes it tight. Here, any centers are exact and k-means makes them tight. The maximum of lower bounds is a lower bound, so the two families compose freely in one cascade.

A note on terminology. Throughout this paper, each pruning step of the cascade is called a *level*. The *ball level* applies the ball bound (2) to all points. Each *PCA level* applies the prefix bound of Lemma 3.1 at one width. A query passes through the levels in order, so we speak of a level's *survivors*: the candidates that its bound could not discard.

3.5 Complementarity

The PCA truncation bound is tight when the eigenvalue spectrum decays steeply. This is a *global, linear* property (Figure 1, left). The ball bound is tight when points sit close to their centroids relative to the spread of $\lVert q - c \rVert$. This is a *local* property. It holds for multimodal data, and more generally for data of low intrinsic dimension, where d_p and the centroid distances spread widely instead of concentrating. The two conditions are different, not exclusive: data can satisfy both or neither (i.i.d. noise, §6.4). A synthetic dataset of 512 tight clusters whose centers span all dimensions shows both sides at once. Its spectrum is nearly flat (at $d = 128$, 95% of the variance needs 110 of 128 dimensions), yet the PCA bound is not loose there: the 8-D level alone leaves 0.2% of the points, because pruning is a tail event (§4). What limits the PCA cascade to 2.1× ($n = 200$k, $d = 64$) is the cost of that first level, eight multiply-adds for every point. The ball bound prunes the same data to 0.2% with one subtraction per point, before a single projection is computed.

4 THE SELF-CONFIGURATION FUNNEL

The funnel answers, from a sample and in seconds, "which levels should this index have for *my* data?" Figure 2 shows its shape: two statistical tiers that may only *nominate*, and one measured tier that alone *decides*.

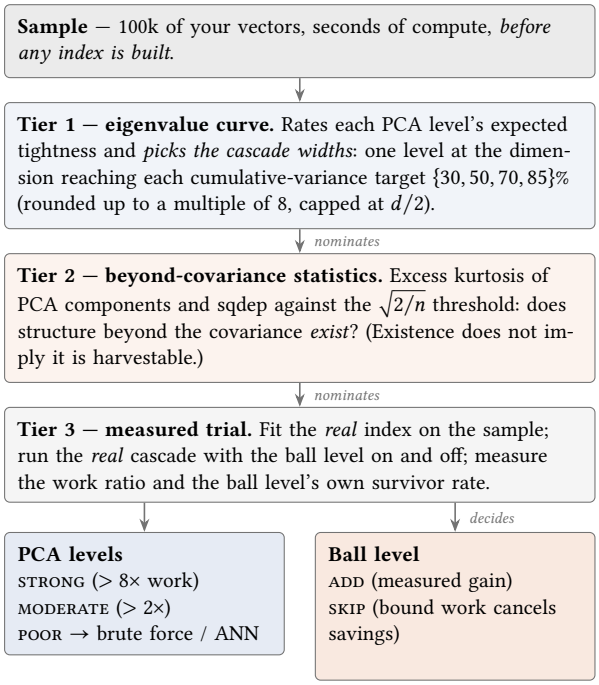


Figure 2: The self-configuration funnel. Tiers 1–2 are cheap statistics, and they may only *nominate*. Tier 1 also derives the cascade widths from the eigenvalue curve. Tier 3 is a measured trial of the real cascade, and it alone *decides*. Section 6.5 shows that the sample verdicts hold at 10^6–10^7 scale. The tier-2 mispredictions in the text (SIFT and latent-8, in opposite directions) are why no statistic can replace the trial.

Tier 1 — eigenvalue decay, which also *picks* the widths. Fit PCA on a sample. The eigenvalue curve is not merely a heuristic signal: in the PCA basis the coordinates of the difference between two independently drawn points are uncorrelated with variances $2\lambda_i$, so for typical candidate pairs

$$\frac{\mathbb{E}\, L_2^2\big(T_k(X'), T_k(Y')\big)}{\mathbb{E}\, L_2^2(X, Y)} = \frac{\sum_{i \le k} \lambda_i}{\sum_i \lambda_i}, \tag{3}$$

which is the cumulative explained variance at k. The curve therefore reads directly as the ratio of the expected squared width-k prefix distance to the expected squared full distance, over the population the cascade prunes. In this sense, each dataset's curve is its prunability signature (Figure 3). Two blind spots follow from the same statement, and each one motivates a later tier. First, Equation (3) is an expectation over pairs, but pruning is a tail event: a candidate is discarded only when its prefix distance clears the radius. Scalar summaries of the curve can therefore mislead. For example, GIST1M needs 303 of 960 dimensions for 95% variance, which looks like a flat spectrum by the usual standard. However, its steep *head* (48% by eight components) is what the early levels actually use, and the measured verdict is STRONG. Second, the spectrum captures only second-order structure. Clustered structure that no rotation can expose is invisible to it. Tier 2 screens for that structure, and tier 3 settles the question.

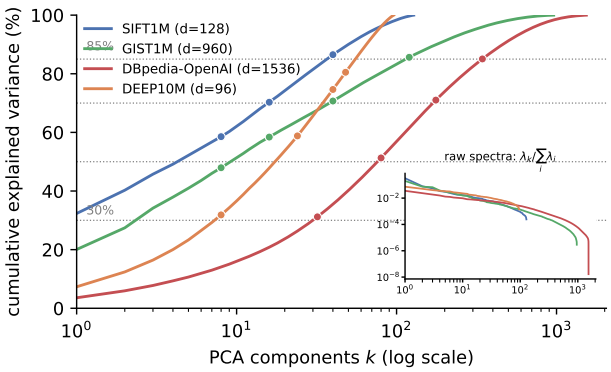


Figure 3: Cumulative explained variance on 100k samples, with components on a log scale. Each curve is a dataset's prunability signature: by Eq. (3), the curve is the ratio of expected squared prefix distance to expected squared distance at width k. Dots mark the widths the funnel selects at the $\{30, 50, 70, 85\}$% targets (dotted lines); targets that round to the same width are merged, so SIFT gets three levels. SIFT and GIST have steep heads. DBpedia's curve is shifted to the right, which is why a fixed d=128 ladder misreads it as unprunable. Relative to its dimension, DEEP is the flattest, which matches its MODERATE verdict; its last width sits below the 85% line because of the $d/2$ cap. The inset shows the raw spectra (per-component variance fraction, log–log), for readers who want the decay rates directly.

Preliminary versions of this work used a fixed ladder $(8, 16, 32, 64)$. We discovered that this ladder is silently calibrated to $d \approx 128$. On 1536-dimensional OpenAI embeddings, it stops at 4% of the dimensions and prunes almost nothing: 68% of candidates survive the 64-D level. As a result, the funnel misreads clearly prunable data as hopeless. The fix makes tier 1 constructive. We place one level at the dimension reaching each cumulative-variance target $\{30, 50, 70, 85\}$%, rounded up to a multiple of 8 and capped at $d/2$. We measured this rule against hand-picked ladders on 100k samples, using the work ratio: the fixed $(8, 16, 32, 64)$ ladder on SIFT and GIST1M, and a deeper $(32, 64, 128, 256)$ ladder on DBpedia-OpenAI, where the fixed ladder measures only 1.3×. The rule ties on SIFT (8.2× vs. 8.5×). It *wins* on GIST1M (15.2× vs. 9.3×) and on DBpedia-OpenAI (8.2× vs. 7.6×). A self-configuring index cannot have a hard-coded ladder. The eigen curve nominates the widths, and the trial below still decides.

Verdicts are work ratios, not survivor fractions. We caught a second dimension-calibration bug in the same way. Rating prunability by the final survivor fraction implicitly assumes one dimensionality. GIST1M's 6.9% finalists scored "poor" under a threshold tuned for SIFT. However, at $d = 960$ the same 6.9% means 9.3× fewer terms, because a scan always pays $n \cdot d$. The verdict now derives from the measured work ratio (sample terms vs. $n \cdot d$), which is dimension-aware by construction.

Tier 2 — beyond-covariance statistics. Tier 2 computes two cheap statistics on the PCA-rotated sample. The first is the mean excess kurtosis of the components, which measures marginal non-Gaussianity. The second is sqdep, the mean $|\mathrm{corr}(z_i^2, z_j^2)|$ over component pairs. It measures joint dependence that the PCA rotation does not remove. We compare it against $\sqrt{2/n}$, a conservative noise threshold: for independent components its expected value is about $\sqrt{2/(\pi n)}$, roughly 0.56 times as large. These statistics detect that structure beyond the covariance *exists*. They do not tell whether it is *harvestable*.

Tier 3 — the measured trial. Tier 3 fits the real hybrid index on the sample and runs the real cascade twice, with the ball level on and off. It reports the measured ratio and the ball level's own pruning rate. The verdict is ADD or SKIP.

Tier 3 must be the decider. The evidence is that the statistics mispredict in *both* directions. On SIFT, tier 2 fires: sqdep = 0.032 is 7× the noise threshold, so real beyond-covariance structure exists. Yet the trial says SKIP (1.0×), because 50–59% of the points survive the ball bounds and the bound work cancels the savings. On a latent-dimension-8 Gaussian, tier 2 is clean: kurtosis is near 0 and sqdep sits at the threshold. Yet the trial says ADD (1.9×), because low intrinsic dimension spreads d_p widely and the bounds prune to 23% with zero multimodality. We call the ball level's measured value its *gain*: the cascade's wall-clock time without the ball level divided by its time with it, so a gain above 1 means the ball level helps. Across the eight datasets and trials of Figure 4, this gain falls monotonically as the ball level's own survivor rate rises. Survivors of 0.2–3% give gains of 4.2–6.6×; 15–23% give 1.9–2.7×; 51–59% give parity; and survivors near 100% give 0.8–1.0×, no gain or a measured loss. This is an empirical trend at $d \leq 128$, not a law: the level's cost per point is fixed, while the savings it buys grow with d. The trial measures exactly this quantity. No statistic substitutes for it.

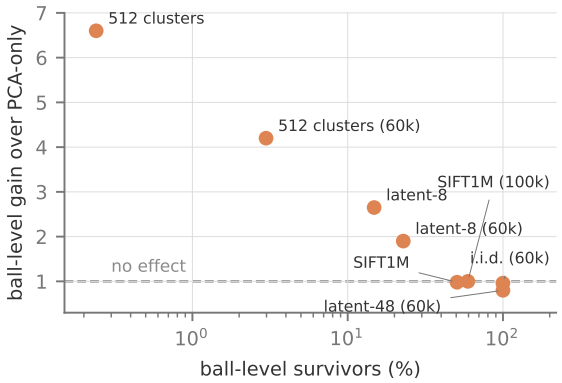


Figure 4: The measured ball-value trend across eight synthetic and SIFT datasets and trials (implementation-parity code): the ball level's wall-clock gain over the PCA-only cascade falls monotonically as its own survivor rate rises. Labels give the sample size where it differs from the full dataset (n = 200k, $d = 64$ for the unlabeled synthetic points; 60k, $d = 128$ for the funnel versions). The funnel's tier-3 trial measures precisely the x-axis quantity, which is why it predicts full-scale outcomes from a sample—including the negative verdicts below the dashed line.

5 THE HYBRID ALGORITHM

5.1 Offline

The offline build has three steps. First, fit PCA; the covariance is accumulated in chunks, so no full `float64` copy of the dataset is ever materialized. Second, select the cascade widths from the eigen curve (§4). Third, fit k-means with MiniBatch (24 s on SIFT1M for k=1024). The default is $k = \mathrm{clip}(\sqrt{N}, 64, 4096)$; the 100k-sample trials use it ($k = 316$), while every full-scale experiment in §6 fixes $k = 1024$, including DEEP10M. For each point we store a cluster id (`int32`) and d_p (`float32`), 8 bytes in total, plus the $k \times d$ centroids and the per-cluster radii. The default comes from a measurement on one dataset. A sweep over $k \in \{8, 64, 256, 1024, 4096\}$ on SIFT1M shows ball-level survivors falling monotonically from 81% to 43%, while fit cost rises from 4 s to 47 s. $\sqrt{N}$ sits at the knee of this gain-versus-cost curve.

5.2 Query Pipeline

(1) k centroid distances ($k \cdot d$ operations).
(2) Ball bounds (2) for all N points: one gather and one absolute difference each.
(3) **Probe**: exact distances to the PROBE= 50 points with the smallest ball bounds seed a safe initial radius r (the Top-50 probe of §3.1, seeded by ball bounds instead of coarse projections).
(4) Ball pruning: retain points with bound$^2 < r^2$.
(5) PCA cascade on the survivors at the selected widths ((8, 16, 32, 64) on SIFT1M), with a **re-probe** at the first PCA level: before pruning, exact distances to the 50 best 8-dimensional candidates tighten r. Without this step the ball-seeded radius is loose and the cascade degrades (8-D survivors 27.2% versus 14.6% with it); with it, the cascade matches PCA-only tightness level-for-level.
(6) Full-distance check on the final survivors. In exact arithmetic the result is identical to brute force. It is additionally verified against brute force, and against the official ground truth where one exists.

Floating point. The exactness argument holds in exact arithmetic. In `float32`, rounding can make a computed bound exceed a computed distance by a few units in the last place. The ball bound is the most exposed: for near-duplicate points, $\|q-c\|$ and d_p nearly cancel. Near-ties can therefore in principle resolve differently from a brute-force scan. Inflating the radius by each bound's rounding-error estimate before the pruning test closes this gap at negligible cost. The results reported here use the unmodified test, and they matched brute force on every query we checked.

5.3 Exact k-NN

For k-NN, the pruning radius generalizes from the best evaluated distance to the k-th smallest exact distance among the points evaluated so far. The k-th order statistic over a *subset* can only overestimate the true one. Therefore, any point whose lower bound reaches this radius provably cannot enter the k-NN set. One subtlety carries an exactness bug if it is missed. The evaluated multiset can contain a point twice, because the probe and re-probe sets overlap. The k-th order statistic of a multiset with duplicates is biased low, and a radius that is too small *over*-prunes. The minimum (the 1-NN special case) is not affected by duplicates, but the k-th order statistic is. The radius must therefore be taken over *distinct* evaluated points. The probe size also grows to $\max(50, k)$, so that the initial radius is a valid k-th statistic. For $k \geq 50$ that statistic is the probe's maximum, the loosest valid choice; §6.3 shows what this costs at $k = 100$. Correctness is verified against brute force across datasets, both cascade paths, and k up to $n - 1$. A second subtlety surfaced during verification. Integer-valued descriptors make exact distance *ties* real: SIFT query 13 has two points at squared distance exactly 61334, and only one of them is in the official ground truth. k-NN sets are therefore unique only up to ties at the boundary.

5.4 Out-of-Core Layout

Pruning fragments I/O only if the layout ignores it. This is the classic index-versus-scan crossover: hundreds of scattered small reads can lose to one full sequential scan. Our two-tier design keeps *resident* only what pruning decisions touch. The resident tier holds the rotated sketch of the first max(levels) PCA dimensions, the ball metadata (`assign`+d_p, 8 bytes per point), an 8-byte row index into the cluster-major file, the centroids, and the per-cluster offsets. On our four datasets this is 1.8–7.7× smaller than the data (Table 8). Full-dimensional vectors live on disk in two interchangeable files. The first keeps the original order, so each fetch is one small aligned read. The second is *cluster-major*: each k-means cluster is a contiguous, page-aligned block (under 0.4% padding), written sequentially into `fallocate`-preallocated extents. The entire cascade (probe, re-probe, and all pruning levels) runs on the resident tier. The disk is touched only by the fetch set: probe, re-probe, and finalists. Section 6.6 measures exactly which points that is.

Two structural facts make the layout matter. First, the cluster-level bound alone is a weak block-skipper: on SIFT1M, 40–52% of clusters survive it. Second, the fetch set is far more concentrated than uniform. The nearest neighbors of a query are near each other, so they share clusters, and the final fetch becomes a few large reads. This is the wisdom of the VA-file [14], SPANN [2], and DiskANN [13]: scan small sketches sequentially, and fetch survivors in a layout-aware way. The same k-means that supplies the bound also supplies the layout. Section 6.6 measures when this pays: the cluster-major file beats per-point fetches only on DEEP10M, where the fetch set is largest.

6 EVALUATION

Metrics policy. Primary claims are stated in hardware-independent work units. The reason is that wall-clock proved regime-dependent: the same algorithm measured 4.7×, 3.6×, 2.7×, and parity under changes of numeric type, machine load, and implementation tuning, while its work counts never moved (details below). The primary metrics are: (1) distance terms per query, with the pruning machinery's own work charged separately as bound-ops; (2) full-vector fetches and bytes read from storage per query, which are the out-of-core I/O currency; (3) resident RAM bytes per vector; and (4) the survivor cascade, which is the algorithmic fingerprint. Wall-clock appears as a check on a first-order cost model ($t \approx$ reads · latency + bytes/bandwidth + ops/throughput), with FAISS supplying the tuned-constants reference. A reader can therefore plug their own machine's constants into our work numbers and predict their own outcome.

Table 2: SIFT1M, 100 official queries, medians of 5 repetitions, implementation parity. All methods return the ground-truth neighbor on every query.

Method	ms/query	Speedup	Survivors per level (%)
brute (NumPy)	167.3	1.0×	—
PCA-only	61.1	2.7×	14.7 / 5.2 / 0.8 / 0.06
hybrid	62.2	2.7×	ball 50.6 → 14.6 / 5.1 / 0.7 / 0.05
FAISS-flat, 1 thread	29.3	5.7×	(scans 100%)
FAISS-flat, batch-100	5.9	28×	(scans 100%)

Setup. Intel i9-9900K (8C/16T, AVX2+FMA, no AVX-512), dual-channel DDR4, 30 GB usable RAM; NVMe SSD measured under `O_DIRECT` at 2.04 GiB/s sequential (8 MiB reads) and 80 µs per random 4 KiB read; `faiss-cpu` 1.15 dispatching its AVX2 build; NumPy and OpenBLAS at the AVX2 tier—baseline and FAISS use the same instruction set. SIFT1M in-RAM timings (Table 2) are medians of 5 repetitions on a quiet machine with warm cache. The multi-dataset and k-NN tables report mean ms/query over 100 queries in a single repetition on a quiet machine. The out-of-core table reports median ms/query and mean bytes and reads (Table 8). Work counts and byte counts are exact and run-invariant; only the millisecond columns carry run noise. In RAM, exactness is verified on every query of every experiment—against in-harness brute force, and against official ground truth where one exists. Out of core, the point and block arms are checked against the scan arm on its 10 queries. Results are CPU-governor-insensitive (medians within 2%).

6.1 SIFT1M

Table 2 gives the in-RAM picture, and Figure 5 shows the cascade behind it. The PCA cascade alone reaches 2.7×. **The ball level is a wall-clock tie on SIFT**: halving the candidates saves about as much as its bound computation costs. This is exactly what the sample trial predicts (SKIP, §4). On SIFT, the hybrid's value is therefore limited to the out-of-core and residency settings (§5.4). The wall-clock case for the ball level rests on clustered and low-intrinsic-dimension data (§6.4). The survivor cascade exactly reproduces the preliminary reports'.

Table 3 states the same experiment in the primary currency. Three facts that the wall-clock hid now become explicit. First, the cascade's *algorithmic* reduction is 10.2×. This number is machine-free, and it was invariant across every run in this paper, while the

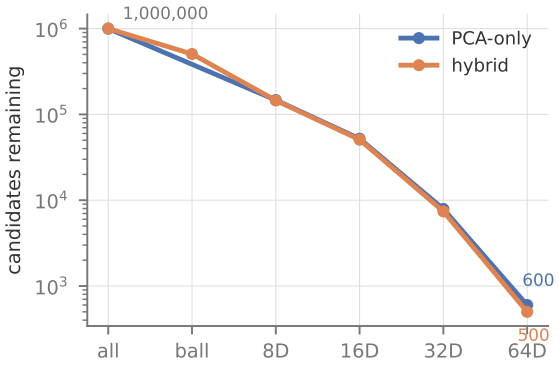


Figure 5: Candidates remaining after each level on SIFT1M (log scale, average over 100 official queries). The cascade discards four orders of magnitude before any full-distance evaluation; the hybrid enters the PCA levels with half the population and finishes equally tight. This profile is the algorithmic fingerprint: it was invariant under every change of numeric type, machine load, and implementation tuning in this paper.

Table 3: Hardware-independent work per query on SIFT1M. Terms are per-dimension $(x-y)^2$ evaluations; bound ops are per-point ball-bound evaluations, charged separately. Counts derive analytically from the survivor statistics (`count_ops` in the repository)—an exhaustive scan is $N \cdot d$ terms by definition, so no instrumentation of compiled libraries is needed. These columns are identical on every machine and every run.

Method	Terms (M)	Bound ops (M)	Terms vs brute
brute / FAISS-flat	128.0	—	1.0×
PCA-only	12.60	0	**10.2**×
hybrid	8.67	1.00	14.8×

millisecond columns wobbled with load. Second, FAISS-flat performs 10–15× more work than the cascade and still wins the in-RAM wall-clock. The separation between engineering and algorithm is now quantitative rather than rhetorical. Third, the hybrid cuts terms further (12.6 → 8.7 M) but adds 1.0 M bound operations of a different kind: gathers rather than streaming reads. This is why its wall-clock ties PCA-only despite the smaller term count. It is also why work accounting that omits the pruning machinery would mislead. Both counts charge each level for its full prefix, because the implementation recomputes it from the first coordinate. Accumulating only the newly added coordinates would, at the same survivor counts, cut the terms to about 10.3 M (PCA-only) and 6.4 M (hybrid).

FAISS `IndexFlatL2` wins the in-RAM race outright. It scans all 10^6 vectors in 29.3 ms on a single thread. The gap is fusion, not SIMD. Both sides run AVX2, but FAISS's kernel makes one pass over the data (~512 MB of traffic per query), while NumPy materializes temporaries (~3 passes, 1.5–2 GB). That is a 5.7× advantage

Table 4: The exactness–speed–work–memory trade-off on SIFT1M (fp32, $10^6 \times 128$). The ms/query column is measured in RAM. Flat scans must hold every vector resident (512 B/vector). The cascades can instead keep only their sketch resident and page full vectors from storage (§5.4, Table 8): the 64-dimensional sketch of the $(8, 16, 32, 64)$ ladder costs 256 B/vector, and the ball metadata costs 8 B/vector. Parentheses give the RAM saving relative to a flat scan. The hybrid's work is written as distance terms plus ball bound-ops (as in Table 3); the two are kept separate because a bound-op is cheaper than a term, so they are not summed. The last row situates disk-resident ANN systems. Their residency (taken from their own reports, not measured here) is lower than the cascades', but they answer the approximate problem.

Method	Exact	ms/query	Terms (M)	Resident RAM
FAISS-flat, batch-100	yes	5.9	128.0	512 MB (1×)
FAISS-flat, 1 thread	yes	29.3	128.0	512 MB (1×)
brute (NumPy)	yes	167.3	128.0	512 MB (1×)
PCA-only	yes	61.1	12.6	256 MB (2.0×)
hybrid	yes	62.2	8.7 +1.0	264 MB (1.9×)
IVF / disk ANN [2, 8, 13]	no	—	—	~8–64 MB

at identical work. This comparison scopes our claim rather than defeating it. The cascade touches 1,700× fewer full vectors. Out of core it keeps 272 bytes per vector resident (the 64-dimensional sketch plus per-point metadata), against 512 for a flat scan that holds the data in RAM. In addition, its bounds compose with FAISS-grade kernels rather than compete with them: the cascade could drive such a kernel over its survivor sets. Disk-resident ANN systems occupy the same economics but give up exactness. Table 4 summarizes the resulting trade among speed, work, and the machine's most expensive resource.

Why wall-clock is the wrong headline (measured). Two confounds were caught only by asking why ratios changed between runs. The first is numeric type. sklearn's PCA silently outputs `float64`, and the bandwidth-bound brute baseline is 1.86× more expensive in `float64`. A 2 × 2 decomposition on identical code measures the same cascade at 3.6× in f64 and 2.7× in f32. The f64 number matches the first preliminary report's regime and its reported 4.3×. The second confound is implementation parity. An unnecessary full-array gather in our own PCA-only path inflated the ball level's apparent contribution to 2.0×. Fixing it revealed the tie in Table 2. Machine load adds a further multiplier: background bandwidth contention slows the scan-bound baseline the most, which flatters all pruners. Survivor counts were invariant through every one of these changes. This is why we adopted the metrics policy above.

6.2 Four Real Datasets

Table 5 answers a question that SIFT alone cannot: does anything beyond SIFT prune? All four real datasets do. The wall-clock gain over the NumPy scan is larger at high dimension: 2.6× at 128 dimensions against 6.2× at 960 and 6.0× at 1536 (and 1.2× on the

Table 5: Exact 1-NN at full scale, PCA-only cascade. Levels are eigen-curve-selected for GIST1M, DBpedia-OpenAI, and DEEP10M; SIFT1M uses the fixed $(8, 16, 32, 64)$ ladder, which does slightly less work than its eigen-curve levels $(8, 16, 40)$ (12.6 vs. 12.9 M terms). Wall is wall-clock speedup over the NumPy brute-force scan. Terms count per-dimension $(x-y)^2$ evaluations; a scan is $n \cdot d$ by definition. All queries exact.

Dataset	n	d	Wall	Terms (M)	Work red.
SIFT1M	10^6	128	2.6×	12.6	10.2×
GIST1M	10^6	960	**6.2**×	46.1	**20.8**×
DBpedia-OpenAI	10^6	1536	6.0×	119.6	12.8×
DEEP10M	10^7	96	1.2×	244.0	3.9×

Table 6: Exact k-NN, PCA-only cascade, levels as in Table 5: wall-clock speedup over brute force and (in parentheses) work reduction in terms. All queries exact; k=100 additionally matches official ground truth up to boundary ties.

Dataset	$k = 1$	$k = 10$	$k = 100$
SIFT1M	2.6× (10.2×)	1.9× (6.2×)	0.7× (1.6×)
GIST1M	6.2× (20.8×)	4.0× (10.3×)	0.9× (1.5×)
DBpedia-OpenAI	6.0× (12.8×)	3.1× (5.8×)	0.8× (1.2×)
DEEP10M	1.2× (3.9×)	0.8× (2.0×)	0.3× (0.6×)

96-dimensional DEEP10M). Brute force pays the full dimension for every point, while the cascade pays mostly for its first levels. These are gains over the same NumPy baseline as Table 2; we timed FAISS only on SIFT1M. The most important result concerns the modern learned embeddings (DBpedia abstracts under OpenAI `ada-002`). Their flat spectra (664 of 1536 dimensions for 95% variance) looked like the natural failure mode of our approach. In fact, once the cascade widths respect the eigen curve (§4), the out-of-core cascade fetches only 0.10% of the vectors (Table 8). With the *fixed* $(8, 16, 32, 64)$ ladder, the 100k sample measures only 1.3× and reads as unprunable. DEEP10M is the honest boundary case. Its 96 dimensions are already-compressed deep features, and they leave a 3.9× work reduction but only 1.2× in-RAM wall-clock. The funnel's sample verdict (MODERATE, "useful out-of-core") anticipates this, and §6.6 measures its out-of-core value: 2.3–2.6× against our scan, but only 1.1–1.3× against the device's sequential limit. Finally, on all four datasets, the ball level's sample verdict (SKIP everywhere; survivor rates 51–99.9%) was confirmed at full scale. The ball-value trend of §4 holds on real data at 10^6–10^7 points.

6.3 Exact k-NN

Table 6 reports the k-NN generalization (§5.3). The PCA-only cascade wins at $k = 10$ on three of the four datasets (DEEP10M: 0.8×), and it *loses* the in-RAM wall-clock at $k = 100$ on every dataset. Part of the loss is structural: the pruning radius is the k-th best distance, which grows with k, and the survivor counts grow with it. Part comes from the probe. With $\max(50, k)$ probe points, the k=100 radius is the probe's maximum. The hybrid path takes its radius over the probe and the re-probe together, up to $2k$ points. At

Table 7: Structure regimes, implementation parity. The two synthetic rows above SIFT1M have $n = 200$k, $d = 64$; the two below are the funnel's $n = 60$k, $d = 128$ datasets. Latent-m data is an m-dimensional standard Gaussian mapped to d dimensions by a random Gaussian matrix, plus isotropic noise ($\sigma = 0.1$ for $m = 8$, 0.5 for $m = 48$); the clusters have standard-Gaussian centers and $\sigma = 0.05$. "Ball survivors" is the fraction of points the ball level keeps—the quantity the funnel's trial measures.

Dataset	PCA-only	Hybrid	Ball surv.	Verdict
512 tight clusters	2.1×	**13.9**×	0.2%	ADD
latent-8 Gaussian	2.1×	5.6×	15%	ADD
SIFT1M	2.7×	2.7×	51%	SKIP
latent-48 Gaussian	1.9×	0.8× of PCA	100%	SKIP
i.i.d. Gaussian	0.3×	—	99.9%	SKIP (all)

k=100 it beats brute force on GIST1M (1.3×) and DBpedia-OpenAI (1.4×), even though its ball level passes 99–100% of the points, but not on SIFT1M (0.9×) or DEEP10M (0.4×). A larger probe is therefore the first fix for large k. DEEP10M's k=100 work exceeds a scan (0.6×) because each level recomputes its prefix from scratch (§6.1). We have not measured large-k queries out of core.

6.4 Structure Regimes

Table 7 spans the design space, and it deliberately includes the negative results. The flat-spectrum clustered dataset shows where the ball level pays. Its PCA bound is tight in the tail (the 8-D level leaves 0.19% of the points), but that level costs eight multiply-adds for every point, which caps the PCA-only cascade at 2.1×. The ball level prunes to 0.2% before any projection is computed. Its bound requires no multiplications, so it undercuts even the 8-dimensional scan, and the hybrid reaches 13.9×. The latent-8 row works the same way: its 8-D level already leaves almost nothing, and the ball level wins by being the cheaper first filter. Figure 6 makes the contrast with SIFT concrete in the work currency. On SIFT the two cascades do almost the same work, which is the tie: 9.8% of a scan in terms for PCA-only, against 6.8% in terms plus 0.8% in bound-ops for the hybrid. On the clustered data the hybrid does far less work: 12.8% against 0.6% in terms plus 1.6% in bound-ops, so most of its small cost is the ball bound-ops, not distance terms. This is what a survivor-count plot cannot show: both cascades reach the same survivors, but the hybrid reaches them with much less work. On the latent-48 blob, the ball level is measured to *hurt* (0.8×): 100% of points survive, so the level is pure overhead. On i.i.d. noise, the diagnostic correctly reports that nothing works and that brute force or ANN should be used. The i.i.d. row also shows the PCA cascade itself losing to brute force (0.3×); because each level recomputes its prefix, it even does more terms than a scan. That is the honest boundary of the entire approach.

6.5 Extrapolation

Does the sample predict the full dataset? On SIFT, the funnel's trial on a 100k sample returns 1.0× (SKIP), and the full-1M parity benchmark measures exactly that (61.1 versus 62.2 ms). The check

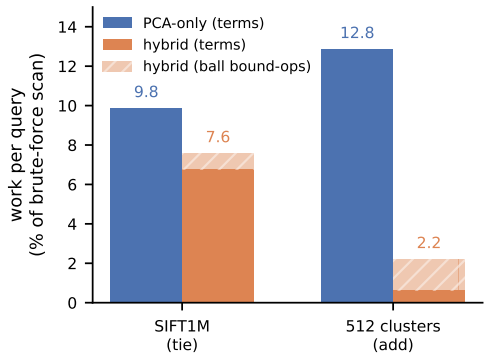


Figure 6: Per-query work of the two cascades, as a percentage of a brute-force scan, on a tie dataset (SIFT1M) and an add dataset (512 clusters). Bars are terms; the hatched segment is the ball level's bound-ops, charged separately and stacked only for display. On SIFT the bars are nearly equal, which is the wall-clock tie. On the clustered data the hybrid does far less work. Both cascades return the same exact result (Table 7); only the work differs, which a survivor-count plot cannot display.

extends to every dataset. GIST1M sampled at 9.3× work with the fixed ladder and at 15.2× with eigen-curve levels; the full-1M runs measured 13.1× and 20.8× respectively, so the sample is conservative. DEEP10M sampled MODERATE at 3.6× and measured 3.9× at 10^7 points. The ball level's sample SKIP was confirmed at full scale on all four datasets. On GIST, for example, 92% of points survive the ball level, the term counts are identical, and the wall-clock is strictly worse. In short, the diagnostic predicts full-scale outcomes from a sample. This includes the negative verdicts, which is its most valuable behavior: it prevents building a useless level.

6.6 Out of Core: Measured Cold Reads

Section 5.4 motivated the two-tier layout from structure. Table 8 now measures it end to end. The protocol is simple: all reads use `O_DIRECT` with page-aligned buffers. The page cache is bypassed, so every query pays device prices. This is a cold-cache protocol with no cache-flushing ceremony. Our scan baseline, however, does not overlap I/O with compute: it reaches 1.0–1.2 GiB/s, against the device's 2.04 GiB/s. A scan pipelined to the device limit would take 234 ms on SIFT1M, 1.75 s on GIST1M and DEEP10M, and 2.8 s on DBpedia-OpenAI. Against that limit, the point arm's speedups are 3.8×, 4.2×, 8.8×, and 1.1×, respectively.

There are three findings. **First, the headline.** On the OpenAI embeddings, exact cold-read 1-NN takes 318 ms against a 5.4-second scan, a 17× speedup (8.8× against the device's sequential limit). It reads 8 MiB instead of 5.7 GiB, which is 730× fewer bytes. The byte ratios are device-independent. How they translate into time depends on how much a device could stream in the time of one random read: about 175 KB on this NVMe drive (80 µs at 2.04 GiB/s), but about 1.5 MB on a disk with a 10 ms seek and 150 MB/s of bandwidth. Seek-bound media therefore favor the

Table 8: Cold-read exact 1-NN under `O_DIRECT` (every byte from the device; page cache bypassed, so each query is cold by construction). Hybrid cascade on the resident sketch, with eigen-curve levels except on SIFT1M, which uses $(8, 16, 32, 64)$ (see text). Resident tier per vector: 272 B (SIFT1M), 496 B (GIST1M), 1,424 B (DBpedia), 208 B (DEEP10M). Scan reads the whole file in 8 MiB chunks and runs the NumPy brute-force kernel on each chunk before issuing the next read. Point fetches each needed vector from the original-order file, one read at a time; block fetches whole clusters from the cluster-major file. ms/query is the median (10 queries for scan, 100 for point and block); MiB/query and reads are means. Point and block match the scan arm's exact results on its 10 queries.

Dataset	Arm	ms/query	MiB/query	Reads
SIFT1M	scan	402	488	62
(488 MiB)	point	**61** (7×)	**2.3** (212×)	588
	block	81 (5×)	37	73
GIST1M	scan	3006	3663	458
(3.6 GiB)	point	**421** (7×)	**91** (40×)	12,379
	block	751 (4×)	1224	158
DBpedia	scan	5385	5860	733
(5.7 GiB)	point	**318** (**17**×)	**8.0** (**730**×)	1028
	block	418 (13×)	374	36
DEEP10M	scan	3574	3664	458
(3.6 GiB)	point	1542 (2×)	**209** (18×)	50,356
	block	**1378** (3×)	780	223

scan and the block arm, not the point arm. **Second, the fetch set is highly cluster-local.** The embeddings' ~1,014 fetched points occupy an average of 27 of 1024 clusters, while uniform placement would touch about 644. SIFT's 583 points occupy 52 clusters. The reason is that the nearest neighbors of a query are near each other. This locality is what makes the cluster-major arm viable at all. Whether it pays depends on the fetch-set size: the block arm beats the point arm only on DEEP10M. **Third, point-versus-block is a device property.** The point arm issues one read at a time. Its scattered small fetches beat the block arm's fewer but heavier reads on three of the four datasets; only DEEP10M's fetch set flips the order. The mean fetch counts in Table 8 are dominated by a minority of far-neighbor queries. The median query fetches 152 vectors on SIFT1M, 2,360 on GIST1M, 120 on DBpedia-OpenAI, and 5,386 on DEEP10M, against means of 588, 12,379, 1,028, and 50,356. This is why the median times sit well below the mean read count times 80 µs. Parallel small reads would extend the point arm's advantage, since NVMe queue depths reward exactly that pattern. Seek-bound media would flip every row toward block. The layout writes both files, so the diagnostic should pick per device.

One design lesson surfaced only here: *cascade depth matters far more out of core than in RAM.* On SIFT1M the eigen-curve rule stops at $(8, 16, 40)$, because the 85% target is reached at 40 dimensions. Those levels fetch 3,381 points. The deeper $(8, 16, 32, 64)$ ladder fetches only 588 and wins the cold-read benchmark (61 vs.

Table 9: ADSampling (faithful NumPy port of the reference implementation, ϵ_0=2.1, Δd=32) versus our PCA-only cascade, $k = 1$. Work in millions of dimension-terms per query; recall under float64-verified ground truth. Ours is exact by construction.

Dataset	ADS terms (recall)	Ours (exact)	Winner
SIFT1M	33.0 (1.000)	**12.6**	ours 2.6×
GIST1M	**44.5** (1.000)	46.1	tie
DBpedia-OpenAI	**49.8** (1.000)	119.6	ADS 2.4×
DEEP10M	325.8 (1.000)	**244.0**	ours 1.3×

91 ms). In RAM the same change saves only 2% of the terms (12.6 vs. 12.9 M), because a finalist costs d terms there but a random read here. The level-selection objective should follow the medium.

6.7 Baseline: ADSampling

ADSampling [5] is the natural adversary. It also prunes with incremental prefix distances after a rotation. However, its rotation is random, so partial distances become unbiased samples, and its stopping rule is a sequential hypothesis test with a tunable error rate ϵ_0. This buys harder pruning per dimension, at the price of the exactness guarantee. For a fair comparison, we ported the reference implementation into the same harness and the same currency: dimensions touched equal terms. Table 9 is honestly mixed in RAM. We win at 96–128 dimensions, ADS wins on the 1536-D embeddings, and GIST is a tie. At k=10, ADS widens its advantage. On recall, we must correct our own first measurement. An earlier float32 ground-truth check miscounted near-duplicate points as recall failures, due to catastrophic cancellation. Under float64 verification, ADS at the default ϵ_0 achieved perfect recall on every k=1 query we ran. It missed one true neighbor in 800 query-tasks (GIST, k=10, recall 0.9990). The distinction that matters is therefore *guarantee versus empirical*, not observed failure. The distinction is structural, because no ϵ_0 turns the test into a bound.

The two methods also differ out of core. ADS pays its $\Delta d = 32$ floor for *every* point in the dataset. SIFT's 33.0M terms and DEEP10M's 325.8M terms sit just above $32n$. ADS therefore touches every vector, and without a resident sketch it would read full-scan bytes regardless of its parameters. With the two-tier layout we give our own cascade, however, its first test could run on a resident 32-dimensional sketch (128 B/vector, less than our 256 B/vector on SIFT1M), and only the candidates that pass would be fetched. We have not measured that configuration. PDX [10] restructures the layout dimension-major, but it still visits every vector. Our cascade fetches only 0.06–1.2% of the vectors (Table 8). Finally, the two methods compose rather than compete: a statistical test could accelerate the final full-distance checks over our survivor sets, at the usual price.

7 LIMITATIONS

High *intrinsic* dimensionality defeats every cheap bound. Distance concentration flattens the eigenvalue spectrum and the spread of d_p at the same time. The funnel exists precisely to detect this case and say so (Table 7, last row). DEEP10M sits near this boundary in RAM. At native-code constants, the in-RAM wall-clock race belongs to fused exhaustive scans (FAISS) at the 128-D scale. ADSampling's statistical pruning also beats our deterministic bounds in RAM on the highest-dimensional embeddings (§6.7). The PCA-only cascade loses the in-RAM wall-clock at $k = 100$ on every dataset we measured; the hybrid path's larger evaluated set recovers 1.3–1.4× on GIST1M and DBpedia-OpenAI, but not on SIFT1M or DEEP10M (§6.3). Our out-of-core scan baseline does not overlap I/O with compute, so speedups against it overstate the gain over a pipelined scan (§6.6). The out-of-core measurements use `O_DIRECT` on datasets that fit in RAM. This is valid device-price accounting, but a larger-than-RAM run, with parallel rather than serial fetch I/O, remains future work. Updates are also future work: the cluster-major layout assumes a static index that is rebuilt periodically, which is the same assumption disk-ANN builds make. Finally, ball-bound and layout quality depend on the k-means sample, as any learned structure does. Correctness never does.

8 CONCLUSION

Exact lower bounds are shapes, and shapes compose. The right composition is a property of the data, and it can be measured on a sample before any index is built. On that principle, this paper delivers an exact k-NN index that configures itself end to end. The cascade widths come from the eigenvalue curve; without this step, 1536-dimensional embeddings masquerade as unprunable. The levels come from a measured trial, and its sample verdicts are confirmed at 10^6–10^7 scale on four real datasets. The breaking points are stated honestly: $k = 100$ in RAM for the PCA-only cascade, DEEP10M's already-compressed features, and i.i.d. data everywhere. The out-of-core argument has become a measurement: exact cold-read queries run 2–17× faster than our disk scan (1.1–8.8× against the device's sequential limit) and read 18–730× fewer bytes. The recurring lesson of this work is simple. Every constant we had silently tuned to one dataset—the cascade widths, the verdict thresholds, even our own ground-truth checker—failed on the next dataset. Every fix was the same: derive the constant from a measurement. Measure the data to choose the index; count the work to predict the time.

Artifact. Code, diagnostics, and all experiment scripts:
`github.com/taiwuchiang-gmail/hierarchical-pca-search`

REFERENCES

[1] Sergey Brin. 1995. Near Neighbor Search in Large Metric Spaces. In *VLDB*.
[2] Qi Chen, Bing Zhao, Haidong Wang, Mingqin Li, Chuanjie Liu, Zengzhong Li, Mao Yang, and Jingdong Wang. 2021. SPANN: Highly-Efficient Billion-Scale Approximate Nearest Neighbor Search. In *NeurIPS*.
[3] Charles Elkan. 2003. Using the Triangle Inequality to Accelerate k-Means. In *ICML*.
[4] Christos Faloutsos, M. Ranganathan, and Yannis Manolopoulos. 1994. Fast Subsequence Matching in Time-Series Databases. In *SIGMOD*. 419–429.
[5] Jianyang Gao and Cheng Long. 2023. High-Dimensional Approximate Nearest Neighbor Search: with Reliable and Efficient Distance Comparison Operations. *Proceedings of the ACM on Management of Data* 1, 2 (2023).
[6] Tiezheng Ge, Kaiming He, Qifa Ke, and Jian Sun. 2013. Optimized Product Quantization for Approximate Nearest Neighbor Search. In *CVPR*.
[7] Aristides Gionis, Piotr Indyk, and Rajeev Motwani. 1999. Similarity Search in High Dimensions via Hashing. In *VLDB*. 518–529.
[8] Jeff Johnson, Matthijs Douze, and Hervé Jégou. 2019. Billion-Scale Similarity Search with GPUs. *IEEE Transactions on Big Data* 7, 3 (2019), 535–547.
[9] Yannis Kalantidis and Yannis Avrithis. 2014. Locally Optimized Product Quantization for Approximate Nearest Neighbor Search. In *CVPR*.

[10] Leonardo Kuffo, Elena Krippner, and Peter Boncz. 2025. PDX: A Data Layout for Vector Similarity Search. *Proceedings of the ACM on Management of Data* 3, 3 (2025).

[11] Yury A. Malkov and Dmitry A. Yashunin. 2018. Efficient and Robust Approximate Nearest Neighbor Search Using Hierarchical Navigable Small World Graphs. *IEEE Transactions on Pattern Analysis and Machine Intelligence* 42, 4 (2018), 824–836.

[12] Stephen M. Omohundro. 1989. *Five Balltree Construction Algorithms*. Technical Report TR-89-063. International Computer Science Institute.

[13] Suhas Jayaram Subramanya, Fnu Devvrit, Harsha Vardhan Simhadri, Ravishankar Krishnaswamy, and Rohan Kadekodi. 2019. DiskANN: Fast Accurate Billion-Point Nearest Neighbor Search on a Single Node. In *NeurIPS*.

[14] Roger Weber, Hans-Jörg Schek, and Stephen Blott. 1998. A Quantitative Analysis and Performance Study for Similarity-Search Methods in High-Dimensional Spaces. In *VLDB*. 194–205.

[15] Peter N. Yianilos. 1993. Data Structures and Algorithms for Nearest Neighbor Search in General Metric Spaces. In *SODA*.